

音楽の数学 — インタラクティブ探求ツール 実行計画書

最終更新: 2026-07-08 / 一連のピボットを経て確定した最終方向性。Claude Code投入用

0. プロジェクト概要

何を作るか：音楽を支配する数学的構造(音響物理・幾何学・群論・組み合わせ論・リズムの数学)を、実際に触って・聞いて理解できるインタラクティブなWebツール。加えて、マイク入力によるリアルタイムのピッチ/リズム検出層を全モジュールに横断させ、「自分の演奏・鼻歌が今どの数学的構造の上にあるか」を即座に示す。

目的：ポートフォリオ。「音楽理論の学習アプリ」でも「作曲家診断ツール」でもなく、確立された数学理論を現代的なインタラクティブ体験として翻訳した、明確に手薄なジャンルを狙う

やらないこと：特定の作曲家・演奏の著作権付き音源解析、演奏採点ゲーム、学習の進捗管理(ユニット制カリキュラム)。これらは全て過去の検討で規約・競合・スコープの理由で外れた

稼働想定：2週間、週23-25h(合計46-50h)。10モジュール全部を同品質で仕上げるには厳しいため、優先順位づけ必須(Part5参照)

対象層：音楽理論に詳しくなくても数学が分かれば楽しめる層、逆に数学に詳しくなくても音で確認できる層。学術寄りだが一般公開に耐えるバランス

1. 意思決定ログ(なぜここに辿り着いたか)

決定	Reasoning
楽器価格比較プロジェクトを完全に破棄	eBay APIライセンス規約(9.4条・9.5条)が、コア機能(複数マーケットの混合表示・価格モデリング)に直結して抵触していたため
作曲家スタイル分析ツールも方向転換	「クラシックのみ」で閉じると規模感が小さく感じられ、「音楽全般」に広げると著作権の壁(参照コーパスが組めない)に再度ぶつかるというジレンマがあった
Spotify/Apple Music連携を断念	両者ともDRM保護により生の音声データを提供せず、規約でなく技術・法律上の壁だったため。メタデータ連携のみに縮小する代替案も検討したが、最終的に本プロジェクトでは音声入力自体が不要になった

演奏ゲーム (Yousician型)を 不採用	月間2000万ユーザー規模の資金調達済み競合が複数存在する、極めて成熟した市場であるため。差別化の勝ち筋が見えなかった
Duolingo型学習 サイトも不採用	musictheory.net(無料・老舗)、EarMaster、AskSia(AI搭載、楽譜アップロード→ドリル自動生成)など、無料～AI搭載の強力な競合が既に存在するため
「音楽を数学的 観点で見たい」 という本音の表 明を機に方向転 換	ここまでの全ての案が「実用ツール」「学習アプリ」という枠で競合と比較されていたが、本来の関心は「数学的構造の探求」だった。これは 3blue1brown/distill.pubに近いジャンルであり、競合の土俵自体が異なる
Tonnetz・ Tymoczkoの幾 何学・フーリエ 解析・群論を採 用	学術的には数十年の蓄積があるが(Euler 1739年のTonnetz、Tymoczko 2006年のScience論文)、モダンなインタラクティブWeb実装は見当たらなかった (Tymoczko本人のツールは2006年頃のMIDI入力デスクトップソフト止まり)
音響物理(倍音 列・純正律・ピ タゴラスのコン マ)を追加	ユーザーが「Hzの比が2/3倍など、数字自体が美しい」という具体例を提示。耳で確認できる実例があることで、抽象的な数学理論に説得力を持たせられる
リズムの数学 (ユークリッドリ ズム・ポリリズ ム)を追加	「なるべく多く」という要望に対し、系統だった追加カテゴリとして提示。 Toussaintの研究など学術的裏付けもある
バックエンドを 撤廃し、フロン トエンド(ブラウ ザ)完結型に変更	今回のテーマは「録音の解析」ではなく「数式に基づく音の生成・可視化」であるため、Web Audio API/Canvas/SVGだけで完結できると判明。ML推論サーバー・DBが不要になり、開発規模が大幅に軽くなった
リアルタイム音 声入力層(ピッチ/ リズム検出)を追 加	「汎用AIにはできないことをやりたい」という要望に対し、正直に切り分けた結果。音楽理論の説明自体は汎用AIでもできるが、マイクからの連続音声ストリームをリアルタイム処理し続けることは不可能。ここが唯一「特化ツールでないとできない」領域だと判断

2. モジュール構成

2.1 全10モジュール(系統別)

#	モジュール	系統	核となる体験
1	倍音列・純正律 vs 平均律	音響物理	純正な五度(3:2)と平均律の五度($2^{7/12}$)を鳴らし比べ、うなりを耳で確認
2	ピタゴラスのコンマ	音響物理	五度を12回積み重ねる操作をシミュレートし、オクターブとのズレ(≈ 1.0136 倍)を可視化
3	Tonnetzエクスプローラー	幾何学	和音格子上をクリックして移動、隣接和音との共通音・声部の動きをアニメーション表示
4	和音の幾何学 (Tymoczko)	幾何学	2音の和音がメビウスの帯を成す空間を可視化、和音間の"最短距離"を声部進行として表示
5	ピッチクラスのフーリエ解析	フーリエ	選んだ音の集合が「どれくらい長調/全音音階的か」をフーリエ係数の大きさとして可視化
6	移調・反転の対称性ブレイグラウンド	群論	二面体群の操作(T/I)を和音に適用し、変化をリアルタイム表示
7	十二音技法	群論	音列を入力し、逆行・反行・逆行反行を自動生成、対称性を可視化
8	最大均等集合	組み合わせ論	12音から7音を均等に選ぶと全音階になる、という定理をインタラクティブに検証
9	ユークリッドリズムジェネレーター	リズム	nビート中にkビートを均等配置するアルゴリズムを可視化+実際に音で再生
10	ポリリズム(LCMグリッド)	リズム	3拍子と4拍子を重ね、最小公倍数グリッド上での一致点を可視化+再生

2.2 リアルタイム音声入力層(横断機能・差別化の核)

機能	技術	連携先モジュール
ピッチ検出	YINアルゴリズム(ブラウザ内JS、軽量DSP、ML不要)	Tonnetz(#3)、和音の幾何学(#4)、フーリエ解析(#5)
キー判定	Krumhansl-Schmuckler key-profile(統計的相関手法)	フーリエ解析(#5)と連動し「今歌っている調」を推定表示
リズムタップ検出	タップタイミングの記録+パターン照合	ユークリッドリズム(#9)、ポリリズム(#10)

Reasoning：この層こそが「汎用AIにはできない」部分。音楽理論の説明や数式の解説自体は汎用AIでも可能だが、連続的な音声ストリームのリアルタイム処理は不可能。ここに機能を集中させることで「専門特化AI」という位置付けが正当化される。

3. 技術スタック(フロントエンドのみ・バックエンド撤廃)

用途	技術	Reasoning
フレームワーク	Next.js(React)またはVite+React	SPA的な操作感が必要で、SSRは必須ではない
音声生成・再生	Tone.js	倍音列・純正律などの周波数を数式通りに正確に生成できる
リアルタイムピッチ検出	Pitchfinder.js(YINアルゴリズム実装、軽量)	ML不要でブラウザ内完結、レイテンシが低い
可視化	Canvas API / D3.js(必要に応じて)	Tonnetz・幾何学空間・フーリエスペクトラムの描画に十分
状態管理	React標準(useState/useReducer)	各モジュールが独立しているため、重い状態管理ライブラリは不要
ホスティング	Vercel(静的ホスティングで完結)	バックエンドがないため、サーバー費用・運用コストがほぼゼロ

Reasoning：DB・API・ジョブキューが一切不要になったことで、環境構築の複雑さが過去のプロジェクト案(eBay連携、Basic Pitch音声解析)から大幅に減っている。

4. 環境構築

4.1 リポジトリ構成

```
music-math-explorer/  
├─ src/  
│   └─ modules/  
│       └─ 01-harmonics-temperament/  
│       └─ 02-pythagorean-comma/  
│       └─ 03-tonnetz/  
│       └─ 04-chord-geometry/
```

```

|   |   |   └─ 05-fourier-pitchclass/
|   |   |   └─ 06-symmetry-playground/
|   |   |   └─ 07-twelve-tone/
|   |   |   └─ 08-maximally-even-sets/
|   |   |   └─ 09-euclidean-rhythm/
|   |   |   └─ 10-polyrhythm/
|   |   └─ lib/
|   |       └─ audio/
|   |           └─ toneEngine.ts          # Tone.jsラッパー、周波数計算
|   |           └─ pitchDetector.ts       # YINアルゴリズムラッパー (Pitchfinder.js)
|   |           └─ theory/
|   |               └─ pitchClassSet.ts   # 集合論・インターバルベクトル計算
|   |               └─ keyProfile.ts      # Krumhansl-Schmuckler実装
|   |               └─ euclideanRhythm.ts # ユークリッドリズム生成アルゴリズム
|   |           └─ viz/
|   |               └─ canvasHelpers.ts
|   └─ components/
|       └─ MicInput.tsx                  # マイク許可・ストリーム取得の共通コンポーネント
|       └─ ModuleShell.tsx              # 各モジュール共通のレイアウト (解説+可視化+音)
|       └─ pages/ または app/
└─ package.json
└─ README.md

```

4.2 package.json(主要依存関係)

```

next (または vite + react)
tone                # 音声生成・再生
pitchfinder         # YINアルゴリズムによるピッチ検出
d3                  # 必要に応じた可視化補助
typescript

```

pipや Python 環境は不要。Node.js環境のみで完結する。

4.3 ローカル起動手順

```

# 1. プロジェクト作成
npx create-next-app@latest music-math-explorer --typescript
cd music-math-explorer

# 2. 依存関係インストール
npm install tone pitchfinder d3

# 3. 開発サーバー起動
npm run dev

```

4.4 事前に人間側で用意する必要があるもの

- Node.js環境(バックエンドが無いため、これ以外に用意するものはほぼ無い)
- マイク許可のテスト環境(モジュール開発時にブラウザのマイク権限を確認)
- (デプロイ時)Vercelアカウント

5. スケジュールと優先順位(46-50h)

10モジュール全部を同品質で作るのは46-50hでは厳しいため、3段階に優先順位を分ける。

Tier 1(必須・MVP、これだけで動くデモとして成立させる)

モジュール	時間	Reasoning
環境構築+共通コンポーネント (ModuleShell, MicInput, toneEngine)	6h	全モジュールの土台
#1 倍音列・純正律 vs 平均律	5h	一番直感的で「聞いて分かる」体験、導入として最適
#3 Tonnetzエクスプローラー	8h	視覚的インパクトが大きく、幾何学系の代表
#5 フーリエ解析	8h	差別化の核。他に類例がない
リアルタイムピッチ検出層(#3, #5と連携)	8h	「専門特化AI」の証明部分。最優先で動かす
小計	35h	

Tier 2(時間があれば追加)

モジュール	時間
#9 ユークリッドリズムジェネレーター	6h
#6 移調・反転の対称性プレイグラウンド	5h
小計	11h

Tier 3(将来の拡張候補、今回は着手しない前提)

- #2 ピタゴラスのコンマ
- #4 和音の幾何学(Tymoczko、実装難易度が高くメビウスの帯の3D表示が必要)
- #7 十二音技法
- #8 最大均等集合
- #10 ポリリズム

Reasoning：Tier1の4項目+ピッチ検出層で「数学的に美しい・触れる・専門AIでないとできない」という3つの柱を証明するには十分。Tier2以降は基盤(ModuleShellやtoneEngine)が完成した後は1モジュールあたりの追加コストが下がるため、時間が余れば量産できる設計にしている。

6. 成功基準

1. Tonnetz上で和音をクリックすると、隣接和音との関係(共通音・声部の動き)が正しくアニメーション表示される
 2. フーリエ解析モジュールで、長調のスケールと全音音階を入力した際に、明確に異なる係数パターンが表示される
 3. マイクに向かって鼻歌を歌うと、リアルタイムでTonnetz上の位置とキー判定が更新される(遅延1秒未満を目標)
 4. 純正律と平均律の五度を聞き比べたとき、うなりの有無が明確に聞き取れる
 5. README/各モジュールに、参照した理論(Tonnetz=Euler 1739、和音の幾何学=Tymoczko 2006など)の出典が明記されている
-

7. 未確定事項

1. Tier1の4項目で確定か、優先順位の入れ替えを行うか
2. マイク入力のレイテンシ実測(YINアルゴリズムの実装次第で体感が変わるため、Day1で早期検証すべき)
3. 各モジュールの解説文の分量・専門用語のレベル感(一般層向けか、ある程度の前提知識を要求するか)